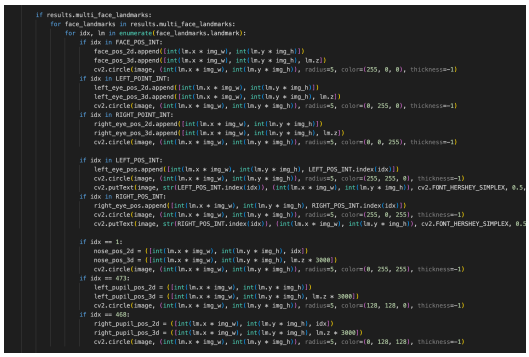
Implementation of a Driver Monitoring System Using MediaPipe

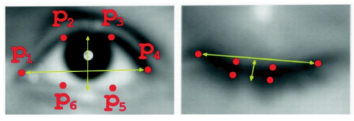
In this report, I describe the implementation of a driver monitoring system based on computer vision techniques, developed using Python and the MediaPipe library. The main objective was to develop a system capable of detecting driver drowsiness and distraction by analyzing camera images in real-time. The system captures and analyzes the driver's eye and head movements, calculating key metrics to determine the level of attention and fatigue, and sends alerts when it detects dangerous behavior. For alerting the user, on-screen warnings were used along with messages sent via Telegram bot for added functionality.

Introduction



After configuring the capture environment, positions of interest from facial landmarks are acquired, which are then used to compute various metrics for our system. This process occurs within a loop, performing analysis for each frame captured by the camera, at a rate of approximately 30 fps.

Methods for Calculating EAR and PERCLOS

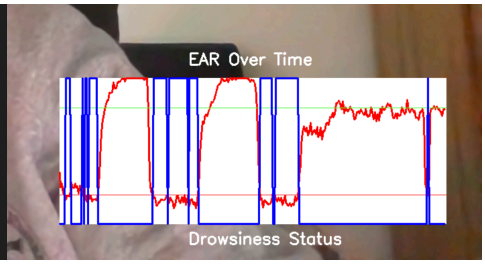


For drowsiness detection, I implemented an algorithm based on the Eye Aspect Ratio (EAR), a metric that quantifies eye openness. I calculate the EAR for both eyes and then average them to obtain a more robust value. Additionally, I normalized the value on a percentage scale to facilitate interpretation and comparison with a threshold of 0.35. Using the EAR values, I constructed a curve showing all EAR values within a 10-second window. I also developed a curve indicating the state when EAR is below the 20% threshold (indicating closed eyes) by assigning a value of 1, and 0 when eyes are open.

$$\text{EAR} = \frac{|Y_2 - Y_6| + |Y_3 - Y_5|}{2 \cdot |X_1 - X_4|}$$

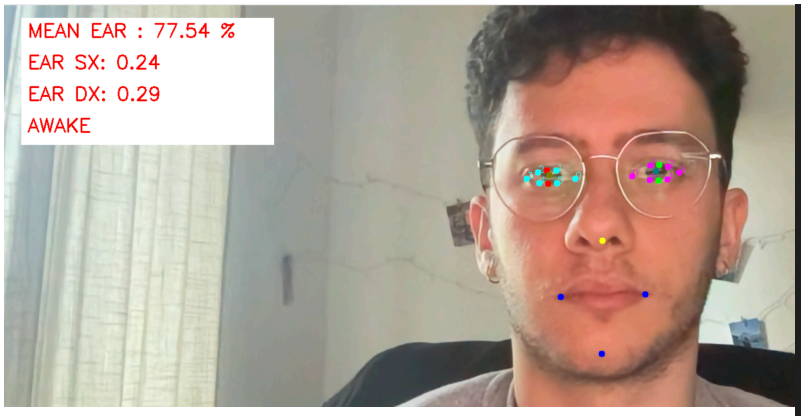
```python
def checkAwake(ear, ears, start, statusIn10s, image, img_w, img_h):
    if ear < 20:
        statusIn10s.append(1)
    else:
        statusIn10s.append(0)

    if len(statusIn10s) > 300:
        statusIn10s.pop(0)
    if st.mean(statusIn10s) > 0.8 and len(statusIn10s) > 300:
        cv2.putText(image, "DROWSY", (50, 200), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)
        if telegram_alert:
            asyncio.run(send_alert("ATTENZIONE: Il conducente sembra assonnato!"))
    else:
        cv2.putText(image, "AWAKE", (50, 200), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)
```



The decision to plot the curves using OpenCV (cv2) instead of Matplotlib was made for efficiency reasons. During my experiments on a Raspberry Pi 4, I observed that using Matplotlib significantly reduced the frame rate. Therefore, for performance optimization, cv2 proved to be the better choice. Using these states, if the average is > 0.8 over 10 seconds, the system sends an alarm message to the driver warning that

they are drowsy. Furthermore, as required, if the EAR > 80% for 10 seconds, the driver is alerted about their inattention.




```
    #stato di t3
    if earsWindow[i] <= 20 and earsWindow[i-1] > 20:
        t3_1 = i
    if earsWindow[i] >= 20 and earsWindow[i-1] < 20:
        t3_2 = i
        if t3_1 != 0:
            t3 = t3 + t3_2 - t3_1
            t3_2 = 0
            t3_1 = 0

    #stato di t2
    if earsWindow[i] <= 80 and earsWindow[i-1] > 80:
        t2_1 = i
    if earsWindow[i] <= 20 and earsWindow[i-1] > 20:
        t2_2 = i
        if t2_1 != 0:
            t2 = t2 + t2_2 - t2_1
            t2_2 = 0
            t2_1 = 0

    #stato di t4
    if earsWindow[i] <= 20 and earsWindow[i-1] > 20:
        t4_1 = i
    if earsWindow[i] >= 80 and earsWindow[i-1] < 80:
        t4_2 = i
        if t4_1 != 0:
            t4 = t4 + t4_2 - t4_1
            t4_2 = 0
            t4_1 = 0
if t4 != 0:
    return (t3-t2)/(t4)
```

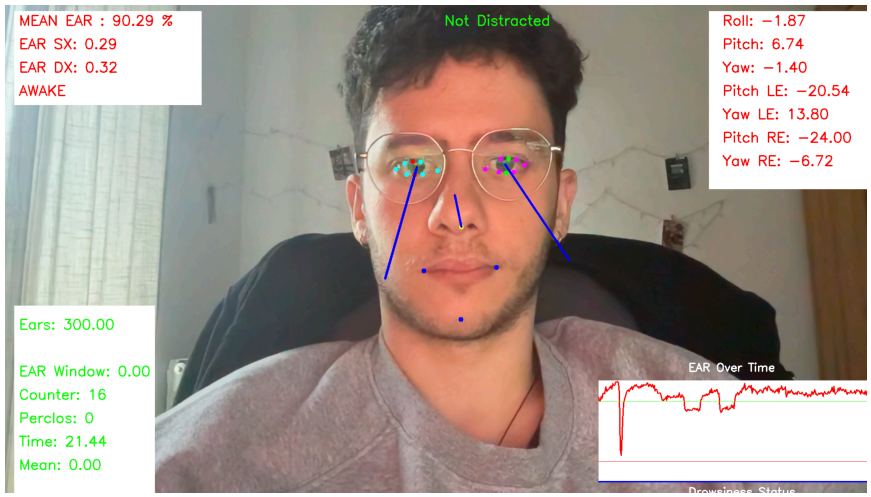
PERCLOS (PERcentage of eye CLOSure) is a standard metric for evaluating drowsiness, measuring the percentage of time the eyes are closed in a given interval. To accomplish this, I wrote a function that calculates the sum of t2, t3, and t4 within the reference window. I consider t1 to be approximately 0 since in a real environment, it is difficult to capture correctly due to noise. Furthermore, the measurement of t2, t3, and t4 is not expressed in seconds; instead, for simplicity, the calculation is based on the number of frames, thus avoiding a simple multiplication by the sampling rate. If the PERCLOS value exceeds a threshold, the system warns the driver that they are falling asleep.

Detection of Gaze Direction and Head Position

To detect driver distraction, I implemented a head position estimation algorithm based on facial landmark analysis. This methodology identifies when the driver's gaze is not directed at the road, indicating potential distractions. To detect the directions I retrive from the landmarks also the z-coordinate. The checkGaze function estimates the orientation of the driver’s head and eyes. It first defines the camera’s intrinsic parameters, such as focal length and optical center, and assumes no lens distortion. Using the 3D coordinates of key facial points (for the face, left eye, and right eye) and their corresponding 2D positions in the image, the function applies a computer

```
def checkGaze(image, face_pos_2d, face_pos_3d, left_eye_pos_2d, left_eye_pos_3d, right_eye_pos_2d, right_eye_pos_3d):
    # The camera matrix
    focal_length = 1 * img_w
    cam_matrix = np.array([ [focal_length, 0, img_h / 2],
    [0, focal_length, img_w / 2],
    [0, 0, 1]])
    # The distorsion parameters
    dist_matrix = np.zeros((4, 1), dtype=np.float64)
    # Solve PnP
    success, rot_vec, trans_vec = cv2.solvePnP(face_pos_3d, face_pos_2d, cam_matrix, dist_matrix)
    success_left_eye, rot_vec_left_eye, trans_vec_left_eye = cv2.solvePnP(left_eye_pos_3d, left_eye_pos_2d, cam_matrix, dist_matrix)
    success_right_eye, rot_vec_right_eye, trans_vec_right_eye =cv2.solvePnP(right_eye_pos_3d, right_eye_pos_2d, cam_matrix, dist_matrix)
    # Get rotational matrix
    rmat, jac = cv2.Rodrigues(rot_vec)
    rmat_left_eye, jac_left_eye = cv2.Rodrigues(rot_vec_left_eye)
    rmat_right_eye, jac_right_eye = cv2.Rodrigues(rot_vec_right_eye)
    # Get angles
    angles, mtxR, mtxQ, Qx, Qy, Qz = cv2.RQDecomp3x3(rmat)
    angles_left_eye, mtxR_left_eye, mtxQ_left_eye, Qx_left_eye, Qy_left_eye, Qz_left_eye = cv2.RQDecomp3x3(rmat_left_eye)
    angles_right_eye, mtxR_right_eye, mtxQ_right_eye, Qx_right_eye, Qy_right_eye, Qz_right_eye = cv2.RQDecomp3x3(rmat_right_eye)
    # Get angles
    pitch = angles[0] * 1800
    yaw = -angles[1] * 1800
    # Define point_RER and point_LEL based on eye landmarks
    point_RER = right_eye_pos_2d[0]
    point_LEL = left_eye_pos_2d[0]
    roll = 180 + (np.arctan2(point_RER[1] - point_LEL[1], point_RER[0] - point_LEL[0]) * 180 / np.pi)
```

vision algorithm (SolvePnP) to determine how the head and eyes are positioned in three-dimensional space relative to the camera. This process yields rotation and translation vectors for the face and each eye. These rotation vectors are then converted into rotation matrices, which are further decomposed to extract the Euler angles: **pitch** (up/down tilt), **yaw** (left/right rotation), and **roll** (side tilt). The function calculates these angles for the head as well as for each eye, providing a detailed description of both head orientation and gaze direction. The **plotDirections** function visually represents the estimated directions of the driver's head and eyes directly on the video frame. For each frame, it draws a line starting from the detected nose position, extending in the direction indicated by the calculated head pitch and yaw angles. If the eyes are sufficiently open (as indicated by the EAR value), the function also draws lines from the positions of the left and right pupils, extending in the directions of their respective pitch and yaw angles.The **check_driver_distraction** function evaluates whether the driver is distracted by analyzing the combined pitch and yaw angles of the head and eyes, as well as the roll angle. If any of these combined angles deviate by more than 30 degrees from the neutral (frontal) position, the system classifies the driver as distracted. When distraction is detected, a warning message is displayed on the screen, and if the distraction persists for more than five seconds, a remote alert (e.g., via Telegram) is sent. The function also tracks the duration of the distraction and this approach ensures that only significant and sustained distractions trigger alerts, reducing false positives and enhancing system reliability.



Conclusion

The implemented Driver Monitoring System meets the assignment requirements by detecting drowsiness and distraction using metrics like EAR, PERCLOS, and head/gaze orientation. It uses MediaPipe for facial landmark detection and OpenCV for image processing, running at 30 FPS.